

Identifying and Categorising Radio Pulsar Candidates with Machine Learning

UE24CS352A Machine Learning – Mini-Project | Team 1, Problem 7 | Yathish (PES1UG22CSXXX) | Code:
https://github.com/Ponakala-Yathish/ML_Mini_Project

1. Problem statement

Radio pulsar surveys produce far more candidate signals than experts can inspect, and most are radio-frequency interference (RFI) or noise. We (i) build a classifier that flags true pulsars, optimising **recall** because missing a pulsar is more costly than reviewing a false alarm, and (ii) cluster the confirmed pulsars to look for sub-groups. The task follows the CS229 report *Application of machine learning methods to identify and categorize radio pulsar signal candidates*.

2. Dataset

HTRU2 (UCI ML repository; Lyon et al., 2016; HTRU survey): 17,898 candidates, of which 1,639 (9.2%) are real pulsars – a severe class imbalance. Each candidate has 8 continuous features: mean, standard deviation, excess kurtosis and skewness of (a) the integrated pulse profile and (b) the DM-SNR curve; label 1 = pulsar. We hold out a stratified 20% test set (3,580 rows) and use the remaining 14,318 for 5-fold cross-validation.

3. Approach

Supervised: Gaussian Discriminant Analysis (baseline; shared covariance, implemented from scratch) versus Random Forest. The Random Forest is (a) scikit-learn's, tuned with a randomized search over `n_estimators`, `max_depth`, `max_features` and `min_samples_split` (best: {"n_estimators": 200, "min_samples_split": 5, "max_features": 4, "max_depth": 110}), and (b) our own NumPy implementation (bootstrap trees, random feature subsets, entropy/information-gain splits). To handle imbalance we randomly oversample the minority class to 50/50 *inside each training fold only*, so validation/test data are never duplicated. The decision threshold is chosen on out-of-fold predictions by maximising TPR-FPR, not on the test set. **Unsupervised:** trained on the pulsars only: K-means (baseline, $k=3$) and a from-scratch Self-Organizing Map (5x5 grid, 100 epochs, exponentially decaying learning rate and neighbourhood radius). Clusters are evaluated with the Silhouette coefficient on all 8 features (raw, as in the paper, and standardized); PCA is used only to visualise clusters and interpret features.

4. Implementation overview

Python 3, NumPy, pandas, scikit-learn, matplotlib. Package `htru/` holds data handling (`data.py`), `gda.py`, `forest.py`, `som.py` and evaluation utilities (`evaluate.py`); `run_supervised.py` and `run_unsupervised.py` reproduce all results and figures, and `make_writeup.py` generates this document from the saved results.

5. Results

Model	CV recall	Test acc.	Test recall @0.5	Threshold*	Test recall @thr	Test prec. @thr	Test AUC
GDA (baseline)	87.4%	97.9%	89.6%	0.19	92.1%	71.2%	0.971
Random Forest (scikit-learn)	86.3%	97.8%	87.8%	0.23	90.5%	80.5%	0.968
Random Forest (from scratch)	87.7%	97.8%	89.9%	0.32	91.8%	78.0%	0.973

*Threshold chosen on out-of-fold training predictions (maximising TPR-FPR). CV recall = mean over 5 folds at threshold 0.5.

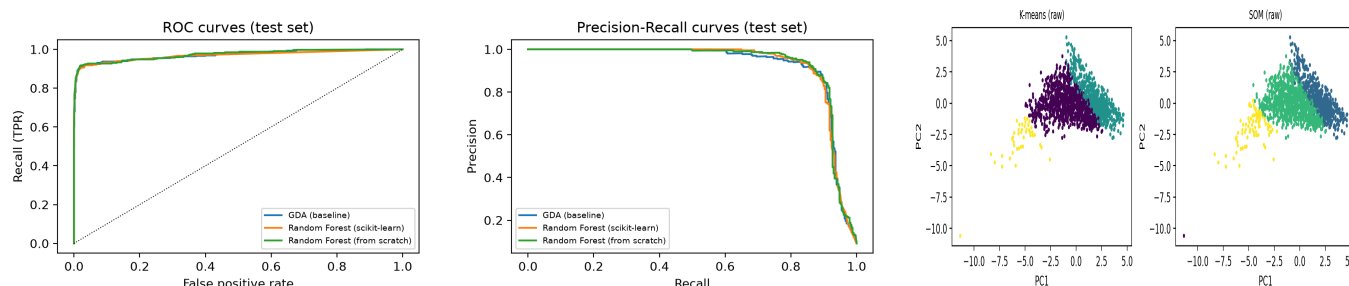


Figure: ROC and precision-recall curves on the test set; K-means/SOM clusters (raw and standardized features) in PCA space.

Clustering (pulsars only)	Features	#clusters	Silhouette
K-means ($k=3$)	raw	3	0.44
SOM (5x5 map)	raw	4	0.44

6. Conclusions

Supervised. On the held-out test set, GDA (baseline) achieved the highest recall (92.1%) at its cross-validated threshold. The tuned scikit-learn Random Forest reached 90.5% recall / 0.968 AUC versus 92.1% / 0.971 for the GDA baseline, and our from-scratch forest reached 91.8% / 0.973. For the scikit-learn forest, moving the threshold from 0.5 to 0.23 changed test recall from 87.8% to 90.5% (precision 87.8% to 80.5%); this trade-off matters because a missed pulsar is costlier than an extra candidate for human review.

Unsupervised. The best K-means silhouette was 0.44 (raw features); values well below 1 indicate that the pulsars do not form clearly separated groups in these 8 summary statistics. This is expected: the features are integrated over frequency/time, so information used to define pulsar sub-classes (e.g. spin period) is not present. **Limitations / future work.** Results come from a single stratified train/test split; the from-scratch forest uses quantile-based candidate thresholds and a fixed configuration; applying the clustering to unprocessed pulse data would be more informative.